Node.js Interview Master Bank

500 Interview Questions for 4 Years Experienced Node.js Developers

Coverage: Node.js, Event Loop, Async JavaScript, Modules, Express.js, REST APIs, Authentication, Security, MongoDB, Mongoose, Error Handling, Performance, Redis, Queues, Microservices, Testing, Deployment & Production.

Node.js Fundamentals

1. What is Node.js?

2. Why is Node.js popular for backend development?

3. What is the Node.js runtime?

4. How does Node.js execute JavaScript?

5. What is the V8 engine?

6. What is libuv?

7. Why is Node.js single-threaded?

8. Is Node.js really single-threaded?

9. What is non-blocking I/O?

10. What is blocking I/O?

11. Synchronous vs asynchronous programming?

12. What are the advantages of Node.js?

13. What are the disadvantages of Node.js?

14. When should you use Node.js?

15. When should you avoid Node.js?

16. Node.js vs Java?

17. Node.js vs PHP?

18. Node.js vs Python?

19. Browser JavaScript vs Node.js?

20. What is the process object?

21. What is process.env?

22. What is process.argv?

23. What is process.cwd()?

24. What is __dirname?

25. What is __filename?

26. What is the Node REPL?

27. How do you check Node.js version?

28. How do you update Node.js?

29. What is npm?

30. What is npx?

31. What is package.json?

32. What is package-lock.json?

33. What are dependencies?

34. What are devDependencies?

35. npm install vs npm ci?

36. What is semantic versioning?

37. What does ^ mean in package versions?

38. What does ~ mean in package versions?

39. What is an npm script?

40. How do you create a Node.js project?

Event Loop & Asynchronous JavaScript

41. What is the Event Loop?

42. Explain the Event Loop step by step.

43. What are Event Loop phases?

44. What is the timers phase?

45. What is the poll phase?

46. What is the check phase?

47. What is the close callbacks phase?

48. What is the microtask queue?

49. What is the callback queue?

50. What is the difference between microtasks and macrotasks?

51. What is process.nextTick()?

52. What is setImmediate()?

53. process.nextTick() vs setImmediate()?

54. setTimeout() vs setImmediate()?

55. What is setInterval()?

56. Can setTimeout(fn, 0) execute immediately?

57. What happens when synchronous code blocks the Event Loop?

58. What is Event Loop starvation?

59. How can you detect Event Loop blocking?

60. How do you prevent Event Loop blocking?

61. What is asynchronous I/O?

62. What is a callback?

63. What is callback hell?

64. How do you avoid callback hell?

65. What is a Promise?

66. Promise states?

67. What is promise chaining?

68. What is async/await?

69. How does async/await work?

70. What happens when an async function throws an error?

71. How do you handle async errors?

72. What happens if you don't use await?

73. Promise.all()?

74. Promise.allSettled()?

75. Promise.race()?

76. Promise.any()?

77. Difference between all and allSettled?

78. How do you run API calls in parallel?

79. How do you run API calls sequentially?

80. How do you handle 100 asynchronous operations?

81. How do you limit concurrent promises?

82. How do you retry failed promises?

83. How do you implement exponential backoff?

84. How do you cancel an async operation?

85. What is AbortController?

86. What happens if a Promise is rejected but not handled?

87. What is an unhandled promise rejection?

88. How do you handle unhandled rejections?

89. What is an uncaught exception?

90. Difference between uncaught exception and unhandled rejection?

Modules

91. What is a Node.js module?

92. What is CommonJS?

93. What are ES Modules?

94. CommonJS vs ES Modules?

95. What does require() do?

96. What does module.exports do?

97. Difference between exports and module.exports?

98. How do you export multiple functions?

99. How do you import multiple functions?

100. What is module caching?

101. Why does Node.js cache modules?

102. How can you clear module cache?

103. What is a built-in module?

104. What is a third-party module?

105. What is a local module?

106. How do you create your own module?

107. What is circular dependency?

108. How do you avoid circular dependencies?

109. What is module resolution?

110. How does Node find a required module?

111. What is package.json main?

112. What is type: module?

113. What is dynamic import?

114. What is lazy loading?

115. What is tree shaking?

116. How do you organize modules in a large project?

117. Controller vs service module?

118. Utility module vs service module?

119. How do you avoid tightly coupled modules?

120. What is dependency injection?

Express.js

121. What is Express.js?

122. Why use Express with Node.js?

123. How do you create an Express server?

124. What is middleware?

125. How does middleware work?

126. What is next()?

127. What happens if next() isn't called?

128. Application-level middleware?

129. Router-level middleware?

130. Built-in middleware?

131. Third-party middleware?

132. Error-handling middleware?

133. What are the four arguments of error middleware?

134. How do you create centralized error handling?

135. How do you handle 404 errors?

136. How do you create routes?

137. What is express.Router()?

138. Why use routers?

139. req.params vs req.query?

140. req.query vs req.body?

141. How do you parse JSON request body?

142. How do you handle form data?

143. How do you handle file uploads?

144. How do you serve static files?

145. What is express.static()?

146. How do you enable CORS?

147. What is CORS?

148. How do you configure CORS securely?

149. How do you validate request data?

150. What validation libraries have you used?

151. How do you sanitize input?

152. How do you handle API errors?

153. How do you create custom errors?

154. What is an error class?

155. How do you handle async errors in Express?

156. How do you log requests?

157. How do you add request IDs?

158. How do you implement API versioning?

159. How do you structure a large Express project?

160. What is MVC architecture?

161. What is layered architecture?

162. Controller vs service?

163. Service vs repository?

164. Why keep business logic out of controllers?

165. How do you implement dependency injection?

166. How do you configure Express for production?

167. How do you handle graceful shutdown?

168. How do you handle uncaught errors?

169. How do you handle large request bodies?

170. How do you set request timeout?

171. How do you protect Express APIs?

172. How do you implement rate limiting?

173. How do you implement pagination?

174. How do you implement filtering?

175. How do you implement sorting?

REST API

176. What is REST?

177. What are REST principles?

178. What is a RESTful API?

179. GET vs POST?

180. POST vs PUT?

181. PUT vs PATCH?

182. DELETE vs POST?

183. What is idempotency?

184. Which HTTP methods are idempotent?

185. What is statelessness?

186. What are HTTP status codes?

187. When do you use 200?

188. When do you use 201?

189. When do you use 204?

190. When do you use 400?

191. When do you use 401?

192. When do you use 403?

193. When do you use 404?

194. When do you use 409?

195. When do you use 422?

196. When do you use 429?

197. When do you use 500?

198. When do you use 503?

199. How do you design REST URLs?

200. How do you version APIs?

201. What is API pagination?

202. Offset vs cursor pagination?

203. How do you implement search?

204. How do you implement filtering?

205. How do you implement sorting?

206. How do you implement field selection?

207. What is API response standardization?

208. What should an API error response contain?

209. What is API idempotency key?

210. How do you prevent duplicate requests?

211. What is rate limiting?

212. What is throttling?

213. Rate limiting vs throttling?

214. How do you handle API timeouts?

215. How do you handle third-party API failures?

216. How do you retry API requests safely?

217. How do you implement API caching?

218. How do you document APIs?

219. What is Swagger/OpenAPI?

220. How do you test REST APIs?

Authentication & Authorization

221. Authentication vs authorization?

222. What is JWT?

223. How does JWT work?

224. JWT structure?

225. What are JWT header, payload and signature?

226. Is JWT encrypted?

227. JWT vs session authentication?

228. Access token vs refresh token?

229. Where should access tokens be stored?

230. Where should refresh tokens be stored?

231. How do you implement login?

232. How do you implement logout with JWT?

233. How do you invalidate JWT?

234. What happens when JWT expires?

235. How do refresh tokens work?

236. How do you rotate refresh tokens?

237. What is token theft?

238. How do you protect tokens?

239. What is password hashing?

240. Hashing vs encryption?

241. Why use bcrypt?

242. What is salt?

243. What is bcrypt salt rounds?

244. bcrypt vs bcryptjs?

245. How do you securely store passwords?

246. How do you implement forgot password?

247. How do you implement reset password?

248. How do you implement email verification?

249. What is role-based access control?

250. What is RBAC?

251. What is permission-based authorization?

252. Role vs permission?

253. How do you create an admin-only API?

254. How do you protect routes?

255. How do you create auth middleware?

256. How do you create role middleware?

257. How do you handle expired tokens?

258. How do you prevent brute-force login?

259. How do you implement account lockout?

260. What is MFA?

261. How would you implement OTP login?

262. How do you secure OTP?

263. How do you prevent OTP abuse?

264. How do you securely store refresh tokens?

265. What is session fixation?

266. What is credential stuffing?

267. What is password spraying?

268. How do you secure authentication APIs?

269. What authentication approach would you choose for a new project?

270. Explain your project's authentication flow end-to-end.

Node.js Security

271. What is XSS?

272. How do you prevent XSS?

273. What is CSRF?

274. How do you prevent CSRF?

275. What is CORS?

276. What is SQL injection?

277. What is NoSQL injection?

278. How do you prevent MongoDB injection?

279. What is command injection?

280. What is prototype pollution?

281. How can prototype pollution happen?

282. How do you prevent prototype pollution?

283. What is Helmet?

284. Why use Helmet?

285. What is HTTPS?

286. Why use HTTPS?

287. How do you secure cookies?

288. What is HttpOnly?

289. What is Secure cookie?

290. What is SameSite?

291. SameSite Strict vs Lax?

292. What is rate limiting?

293. How do you prevent DDoS attacks?

294. How do you protect login APIs?

295. How do you validate uploaded files?

296. How do you prevent malicious file uploads?

297. How do you protect environment variables?

298. Should .env be committed to Git?

299. How do you manage secrets in production?

300. How do you prevent sensitive information in logs?

301. How do you secure API keys?

302. How do you handle dependency vulnerabilities?

303. What does npm audit do?

304. How do you secure npm dependencies?

305. What is dependency confusion?

306. What is supply-chain security?

307. How do you validate request size?

308. How do you protect against ReDoS?

309. What is security middleware?

310. How do you implement security headers?

311. How do you securely handle errors?

312. Why shouldn't you expose stack traces in production?

313. How do you protect admin APIs?

314. How do you audit security events?

315. How would you perform a security review of a Node.js API?

MongoDB

316. What is MongoDB?

317. SQL vs MongoDB?

318. What is a database?

319. What is a collection?

320. What is a document?

321. What is BSON?

322. What is ObjectId?

323. Embedded documents vs references?

324. What is schema design in MongoDB?

325. What is normalization?

326. What is denormalization?

327. When should you embed documents?

328. When should you reference documents?

329. What is an index?

330. Why are indexes important?

331. What is a compound index?

332. What is a unique index?

333. What is a sparse index?

334. What is a TTL index?

335. How do indexes affect performance?

336. What happens if you create too many indexes?

337. What is explain()?

338. How do you optimize a slow MongoDB query?

339. What is aggregation?

340. What is an aggregation pipeline?

341. What is $match?

342. What is $group?

343. What is $project?

344. What is $lookup?

345. What is $unwind?

346. What is $sort?

347. What is $limit?

348. What is $skip?

349. $lookup vs Mongoose populate?

350. What is a MongoDB transaction?

351. When should you use transactions?

352. What are ACID properties?

353. What is read concern?

354. What is write concern?

355. What is replica set?

356. What is sharding?

357. What is MongoDB replication?

358. What is MongoDB Atlas?

359. How do you handle MongoDB connection failures?

360. How do you manage connection pooling?

361. How do you prevent duplicate documents?

362. How do you implement pagination in MongoDB?

363. How do you search MongoDB efficiently?

364. How do you update nested documents?

365. Difference between $set, $push, $addToSet?

366. $deleteOne vs $deleteMany?

367. findOne() vs find()?

368. What is bulkWrite?

369. How do you optimize bulk operations?

370. How would you design MongoDB collections for a large application?

Mongoose

371. What is Mongoose?

372. Why use Mongoose?

373. What is a Mongoose schema?

374. What is a Mongoose model?

375. Schema vs model?

376. What are Mongoose validators?

377. Required vs unique?

378. Does unique: true validate data?

379. What are Mongoose hooks?

380. What is pre() middleware?

381. What is post() middleware?

382. How do you hash passwords using Mongoose hooks?

383. What is populate()?

384. How does populate work?

385. What is lean()?

386. Why use lean()?

387. When should you avoid lean()?

388. What is Mongoose virtual?

389. What are Mongoose getters/setters?

390. What is schema timestamps?

391. What is schema indexing?

392. What is select: false?

393. How do you hide passwords from queries?

394. save() vs create()?

395. updateOne() vs findOneAndUpdate()?

396. What does runValidators do?

397. How do you handle duplicate key errors?

398. How do you handle Mongoose validation errors?

399. How do you handle transactions with Mongoose?

400. How do you optimize Mongoose queries?

401. What is bulkWrite() in Mongoose?

402. How do you implement soft delete?

403. How do you implement audit fields?

404. How do you structure Mongoose models?

405. How would you design a scalable Mongoose application?

Error Handling

406. Why is error handling important?

407. What is a custom error class?

408. How do you create AppError?

409. How do you create centralized error middleware?

410. Operational vs programming errors?

411. How do you handle database errors?

412. How do you handle validation errors?

413. How do you handle JWT errors?

414. How do you handle duplicate key errors?

415. How do you handle third-party API errors?

416. How do you handle timeout errors?

417. How do you handle network errors?

418. What is graceful error handling?

419. How should errors be logged?

420. What information should not be logged?

421. How do you handle uncaught exceptions?

422. How do you handle unhandled promise rejections?

423. Should a Node.js process continue after an uncaught exception?

424. How do you gracefully shut down a Node.js application?

425. Explain your production error-handling strategy.

Performance & Scalability

426. How do you optimize a Node.js API?

427. What causes slow Node.js APIs?

428. How do you detect Event Loop blocking?

429. How do you profile Node.js?

430. What is Node.js profiling?

431. What is memory leak?

432. How do you detect memory leaks?

433. What causes memory leaks?

434. How do you reduce memory usage?

435. What is garbage collection?

436. How does V8 garbage collection work?

437. What is heap memory?

438. What is stack memory?

439. What is a heap snapshot?

440. What are Node.js streams?

441. Why are streams useful for performance?

442. What is backpressure?

443. How do you handle large files efficiently?

444. What are Worker Threads?

445. When should you use Worker Threads?

446. Worker Threads vs Cluster?

447. What is Node.js Cluster?

448. How do you scale Node.js horizontally?

449. What is load balancing?

450. How do you handle 10,000 requests per second?

451. How do you optimize database queries?

452. How do you use caching for performance?

453. How do you reduce external API latency?

454. What is connection pooling?

455. How would you troubleshoot an API that suddenly became slow?

Redis, Queues & Microservices

456. What is Redis?

457. Why use Redis with Node.js?

458. What is caching?

459. Cache-aside pattern?

460. What is cache invalidation?

461. What is Redis TTL?

462. Redis vs MongoDB?

463. What is a Redis lock?

464. How do you prevent cache stampede?

465. What is a message queue?

466. Why use message queues?

467. What is RabbitMQ?

468. What is Kafka?

469. RabbitMQ vs Kafka?

470. What is a background job?

471. How do you process emails asynchronously?

472. How do you implement retry jobs?

473. What is dead-letter queue?

474. What are microservices?

475. Monolith vs microservices?

476. How do microservices communicate?

477. REST vs message-based communication?

478. What is API Gateway?

479. What is service discovery?

480. How would you design a Node.js microservices architecture?

Testing, Deployment & Production

481. What is unit testing?

482. What is integration testing?

483. Unit vs integration testing?

484. What is end-to-end testing?

485. What is Jest?

486. What is mocking?

487. What is stubbing?

488. How do you test Express APIs?

489. How do you test authentication middleware?

490. How do you test database operations?

491. What is code coverage?

492. What is CI/CD?

493. How do you deploy a Node.js application?

494. What is Docker?

495. Why use Docker for Node.js?

496. How do you configure Node.js for production?

497. How do you monitor a Node.js application?

498. What logs and metrics would you monitor?

499. How would you debug a production Node.js issue?

500. Explain your complete Node.js project architecture and one difficult production problem you solved.